

Tutorial: DLL Sideload and function proxying with ShellcodePack

 blog.balliskit.com/tutorial-dll-sideload-and-function-proxying-with-shellcodepack-bddd197c8814

Emeric

December 19, 2025

DLL sideloading is a technique that allows an attacker to have a legitimate signed application run some malicious code on Windows. It works by loading a malicious DLL instead of the intended one by abusing Windows search order. Of course the challenge is to maintain full compatibility and have the malicious DLL proxy the calls to the original DLL.

DLL sideloading is often used as a way to bypass SmartScreen in initial Access context. It can also be useful to evade EDR or application restrictions.

In this tutorial, we'll focus on using ShellcodePack to automate the entire process — from generating proxy DLLs to deploying custom payloads.

Note: This tutorial relies on ShellcodePack command lines. As an alternative you could also perform the same tasks with ShellcodePack GUI.



1. Tutorial Context

The target environment for this tutorial is an up-to-date Windows 11 machine with a running EDR agent.

We'll be sideloading the Autoruns64 application from the Sysinternals Suite. We will first demonstrate a standard DLL sideloading technique, and then we'll explore more advanced function proxying features.

For demonstration purposes, we'll use a simple *MessageBox* payload as our shellcode.

2. Technical Overview

2.1 Understanding Windows DLL Loading

Windows follows a specific search order when loading DLLs:

1. Application calls `LoadLibrary("example.dll")`
↓
2. Windows searches in the following order:
 - Application directory (where the .exe is located)
 - System32 (`C:\Windows\System32`)
 - System directory (`C:\Windows\System`)
 - Windows directory (`C:\Windows`)
 - Current working directory
 - Directories listed in PATH environment variable↓
3. First matching DLL is loaded
↓
4. Import Address Table (IAT) is resolved
↓
5. `DllMain` is called with `DLL_PROCESS_ATTACH`

2.2 DLL Search Order Hijacking Explained

By placing your proxy DLL in a higher priority location, you can intercept the loading process:

- Priority 1: Application directory (Highest priority)
↓
Your proxy DLL placed here
↓
Priority 2: System32 (where the legitimate DLL resides)

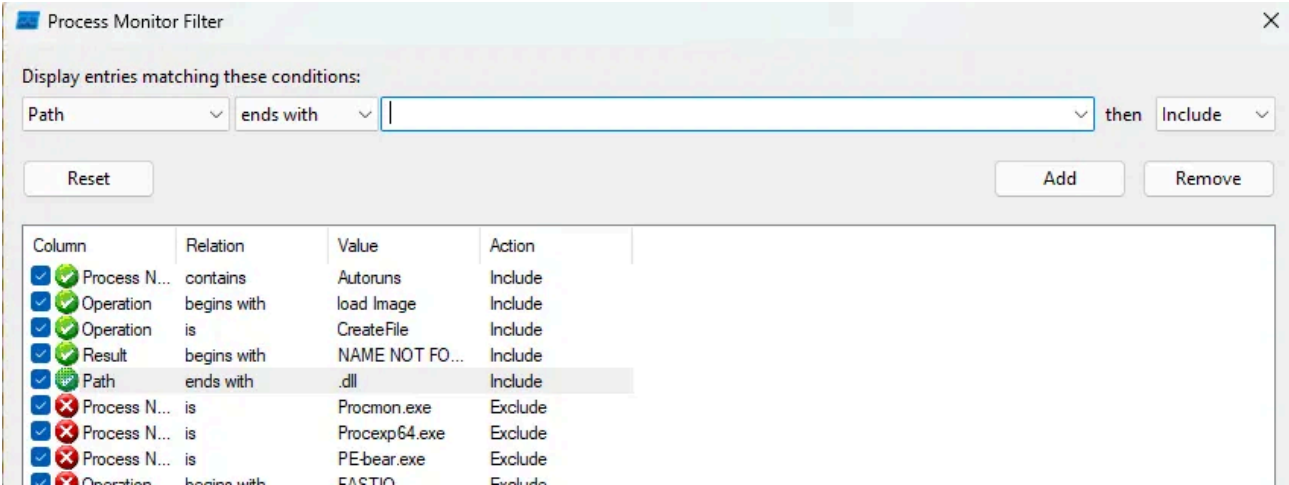
3. Finding a Target Application and DLL

To find a suitable target application for DLL sideloading, you can use tools like [Process Monitor](#) to monitor DLL load events. Look for applications that use the operations Load

Image and CreateFile for DLLs. Then try to identify DLLs that have a result of NAME NOT FOUND, indicating that the application is attempting to load a DLL that doesn't exist in its location (most of the time it will be looking in the current directory, making it a perfect choice).

Here are the filters we used in Process Monitor to find a potential target:

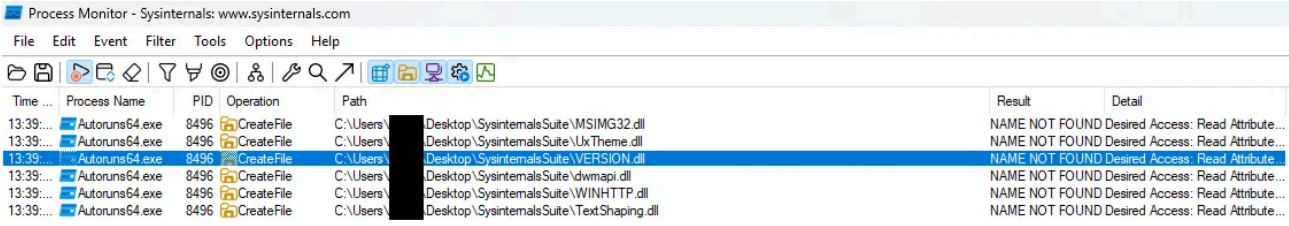
Press enter or click to view image in full size



After analyzing the results, we found that multiple binaries attempt to load version.dll from their own directory. Including most application in program files, such as Chrome, PDF readers, etc. So Version.dll is a perfect candidate as a DLL sideloading vector.

Example: Autoruns64.exe attempting to load several DLLs

Press enter or click to view image in full size



4. DLL Sideloading with Shellcode Pack

4.1: Generate the Payload

To generate a proxy DLL that sideloads into the target application and executes your shellcode, use the following command:

```
shellcode_pack.exe -i "C:\msgbox.bin" -G version.dll --dll-proxy="C:\Windows\System32\version.dll" --arch=WIN_X64 --bypass-profile=resources\bypass_profiles\edr_bypass_profile.json
```

Option breakdown:

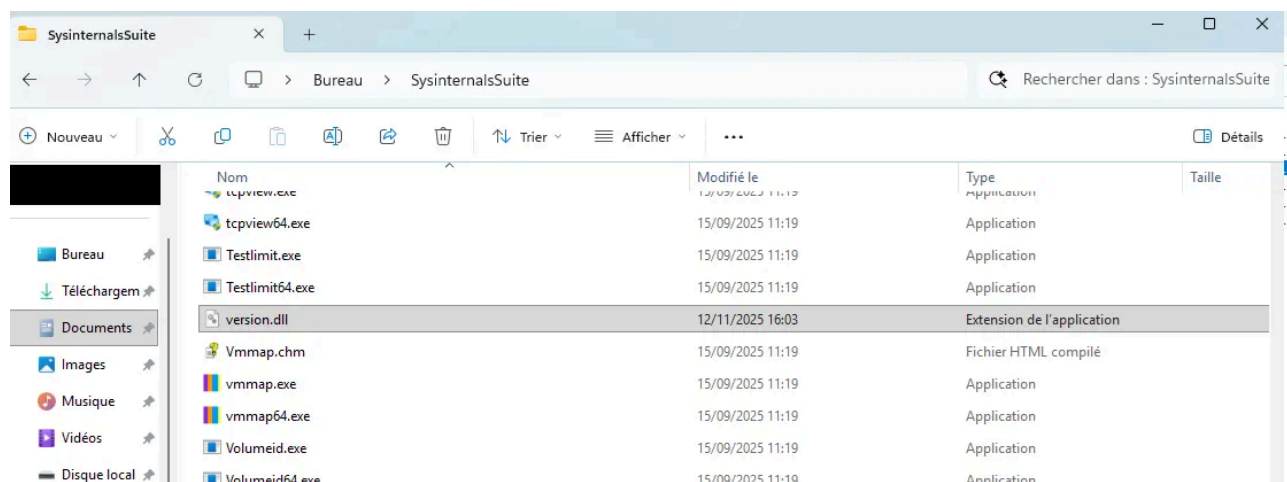
- `--dll-proxy`: Points to the legitimate DLL to forward calls to
- `--arch`: Specifies the architecture of the target application
- `--bypass-profile`: Applies multiple AV/EDR evasion techniques during payload generation

4.2: Deployment Setup

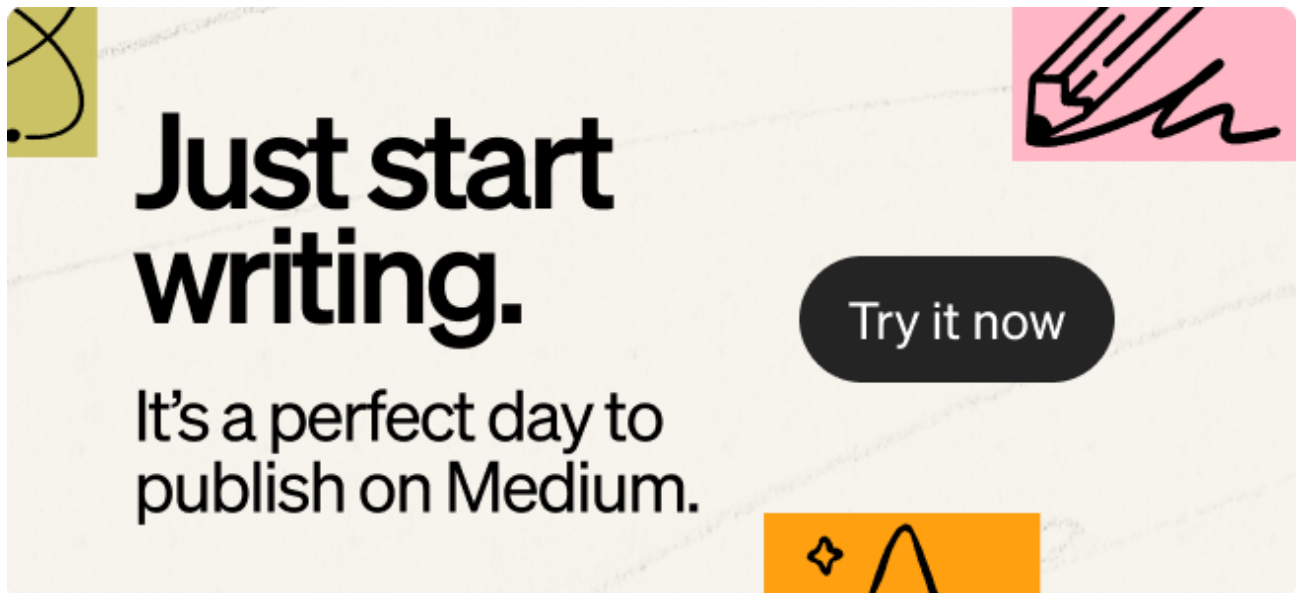
Once you have generated the payload, perform a standard DLL sideloading setup by placing the generated DLL into the application's directory.

As explained in the technical overview, Windows will search for the DLL in the application directory first, before checking System32 where the legitimate DLL resides. This search order behavior is what makes the technique work.

Press enter or click to view image in full size



When the application loads, it will find and load your proxy DLL first, while your DLL forwards legitimate calls to the original DLL in System32 or the location you've set using the `--dll-proxy` option.



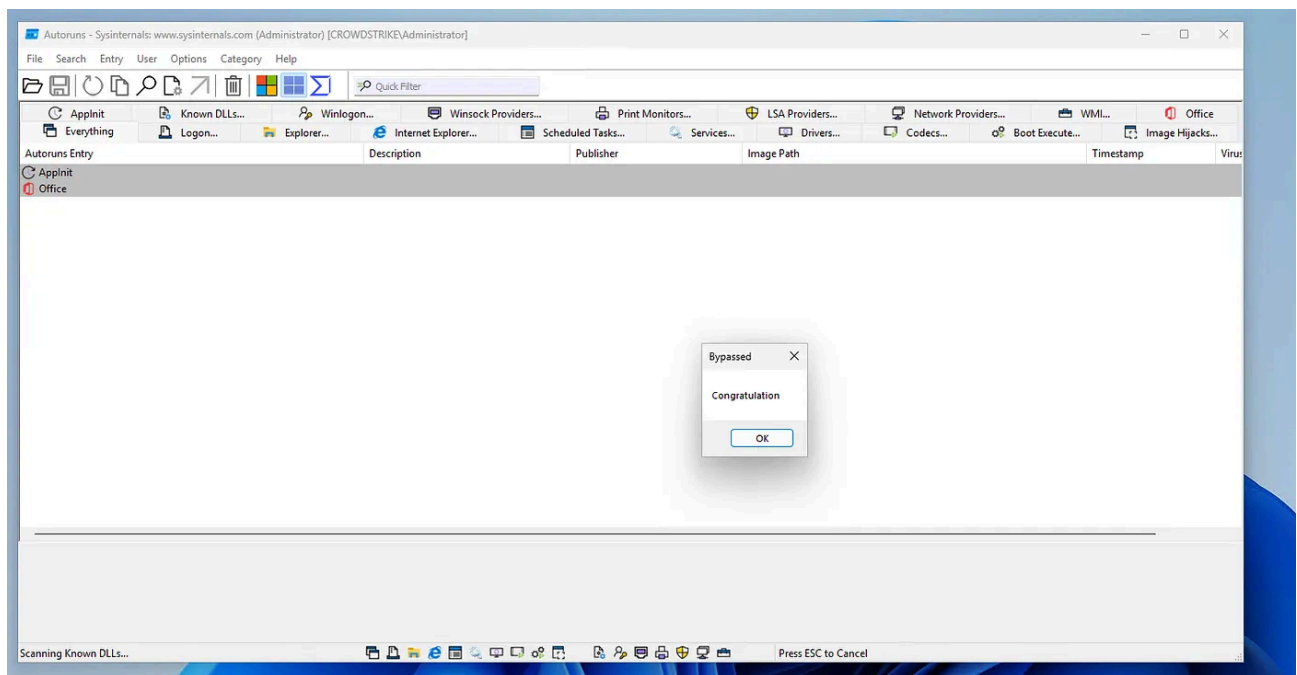
TIP: You can set a relative path for the `--dll-proxy` option to avoid hardcoding system paths, e.g., `--dll-proxy=version2.dll` to reference a DLL in the same directory as your proxy DLL.

Tip2: You may want to export the same function as the original DLL but not forward the calls. For that, use options: `--dll-proxy-do-not-forward`

4.3: Execution

With everything in place, launch the target application `Autoruns64.exe`. You should see your payload execute while the application continues to function normally.

Press enter or click to view image in full size



Additional Note: It might sometimes be useful to increase the reference count so the system assumes the DLL is still used by the process and it will not be unloaded. For that use the next option: `--dll-increase-reference-count`

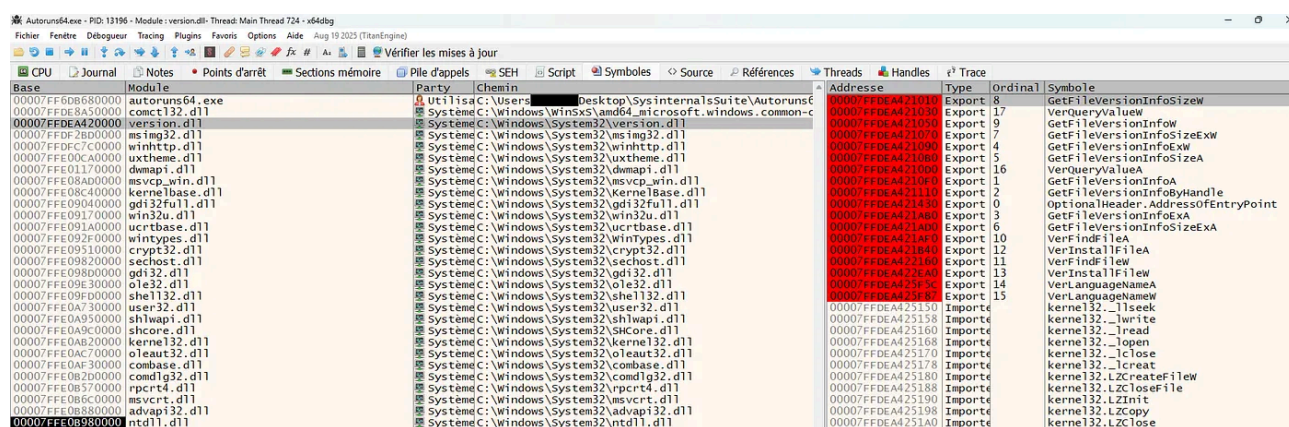
5. DLL Function Proxying

By default, the generated DLL will run the Shellcode from DllMain, as an alternative, you can choose to run the shellcode from specified exported function instead. This is particularly useful if you want to keep it stealthy and only trigger your payload under certain conditions.

5.1: Finding Function Calls

To find out when a specific function is called, you can use debugging tools like [x64dbg](#) or [WinDbg](#) to set breakpoints on the functions of the DLL of interest. In this case, we're interested in the DLL `version.dll`. So we can set breakpoints on the functions exported by the DLL to see when they are called.

Press enter or click to view image in full size



Base	Module	Party	Chemin	Adresse	Type	Ordinal	Symbole
00007FFDB680000	autoruns64.exe	Utilisa	C:\Users\...\Desktop\SysinternalsSuite\Autoruns64.exe	00007FFDB680000	Export	8	GetFileVersionInfoSizeW
00007FFDB680000	conhost.dll	Systeme	C:\Windows\System32\conhost.dll	00007FFDB680000	Export	17	VerQueryValueW
00007FFDB680000	version.dll	Systeme	C:\Windows\System32\version.dll	00007FFDB680000	Export	9	GetFileVersionInfoW
00007FFDB680000	msimg32.dll	Systeme	C:\Windows\System32\msimg32.dll	00007FFDB680000	Export	7	GetFileVersionInfoSizeExW
00007FFDB680000	winhttp.dll	Systeme	C:\Windows\System32\winhttp.dll	00007FFDB680000	Export	4	GetFileVersionInfoExW
00007FFDB680000	uxtheme.dll	Systeme	C:\Windows\System32\uxtheme.dll	00007FFDB680000	Export	5	GetFileVersionInfoSizeA
00007FFDB680000	dwapi.dll	Systeme	C:\Windows\System32\dwapi.dll	00007FFDB680000	Export	16	VerQueryValueA
00007FFDB680000	msvc_p_wins.dll	Systeme	C:\Windows\System32\msvc_p_wins.dll	00007FFDB680000	Export	1	GetFileVersionInfoA
00007FFDB680000	kernelbase.dll	Systeme	C:\Windows\System32\kernelbase.dll	00007FFDB680000	Export	2	GetFileVersionInfoByHandle
00007FFDB680000	gd32full.dll	Systeme	C:\Windows\System32\gd32full.dll	00007FFDB680000	Export	0	OptionalHeader.AddressOfEntryPoint
00007FFDB680000	win32u.dll	Systeme	C:\Windows\System32\win32u.dll	00007FFDB680000	Export	3	GetFileVersionInfoExA
00007FFDB680000	ucrtbase.dll	Systeme	C:\Windows\System32\ucrtbase.dll	00007FFDB680000	Export	6	GetFileVersionInfoSizeExA
00007FFDB680000	wintypes.dll	Systeme	C:\Windows\System32\wintypes.dll	00007FFDB680000	Export	10	VerFindFileA
00007FFDB680000	crypt32.dll	Systeme	C:\Windows\System32\crypt32.dll	00007FFDB680000	Export	12	VerInstallFileA
00007FFDB680000	sechost.dll	Systeme	C:\Windows\System32\sechost.dll	00007FFDB680000	Export	11	VerFindFileW
00007FFDB680000	gd32.dll	Systeme	C:\Windows\System32\gd32.dll	00007FFDB680000	Export	13	VerInstallFileW
00007FFDB680000	ole32.dll	Systeme	C:\Windows\System32\ole32.dll	00007FFDB680000	Export	14	VerLanguageNameA
00007FFDB680000	shell32.dll	Systeme	C:\Windows\System32\shell32.dll	00007FFDB680000	Export	15	VerLanguageNameW
00007FFDB680000	user32.dll	Systeme	C:\Windows\System32\user32.dll	00007FFDB680000	Importe		kernel32...llseek
00007FFDB680000	shlwapi.dll	Systeme	C:\Windows\System32\shlwapi.dll	00007FFDB680000	Importe		kernel32...hread
00007FFDB680000	kernel32.dll	Systeme	C:\Windows\System32\kernel32.dll	00007FFDB680000	Importe		kernel32...lopen
00007FFDB680000	oleaut32.dll	Systeme	C:\Windows\System32\oleaut32.dll	00007FFDB680000	Importe		kernel32...lcreat
00007FFDB680000	combase.dll	Systeme	C:\Windows\System32\combase.dll	00007FFDB680000	Importe		kernel32.LZcreateFileW
00007FFDB680000	condlg32.dll	Systeme	C:\Windows\System32\condlg32.dll	00007FFDB680000	Importe		kernel32.LZcloseFile
00007FFDB680000	rpcrt4.dll	Systeme	C:\Windows\System32\rpcrt4.dll	00007FFDB680000	Importe		kernel32.LZinit
00007FFDB680000	msvcrt.dll	Systeme	C:\Windows\System32\msvcrt.dll	00007FFDB680000	Importe		kernel32.LZcopy
00007FFDB680000	advapi32.dll	Systeme	C:\Windows\System32\advapi32.dll	00007FFDB680000	Importe		kernel32.LZclose
00007FFDB680000	ntdll.dll	Systeme	C:\Windows\System32\ntdll.dll	00007FFDB680000	Importe		

All of the functions from `version.dll` are called during the startup of `Autoruns64.exe`. In our case, we chose the function `VerQueryValueW`, which is called during the startup of `Autoruns64.exe`.

5.2: Generate the payload

To generate a proxy DLL that executes shellcode from an exported function, you need to use two specific options: `--dll-run-shellcode-from-dll-export` and `--dll-export=exported_fct_name`.

Important: For the proxied function to work correctly, the shellcode must return properly to avoid disrupting the host process.

```
shellcode_pack.exe -i "C:\msgbox.bin" -G version.dll --dll-run-shellcode-from-  
proxy="C:\Windows\System32\version.dll" --arch=WINE_X64 --dll-run-shellcode-from-
```

```
dll-export --dll-export=VerQueryValueW --bypass-  
profile=resources\bypass_profiles\edr_bypass_profile.json
```

Option breakdown:

- `--dll-export`: Specifies which function to export from your proxy DLL. It's possible to specify multiple export function (function names separated by a ',')
- `--dll-run-shellcode-from-dll-export`: Executes the shellcode when the exported function is called

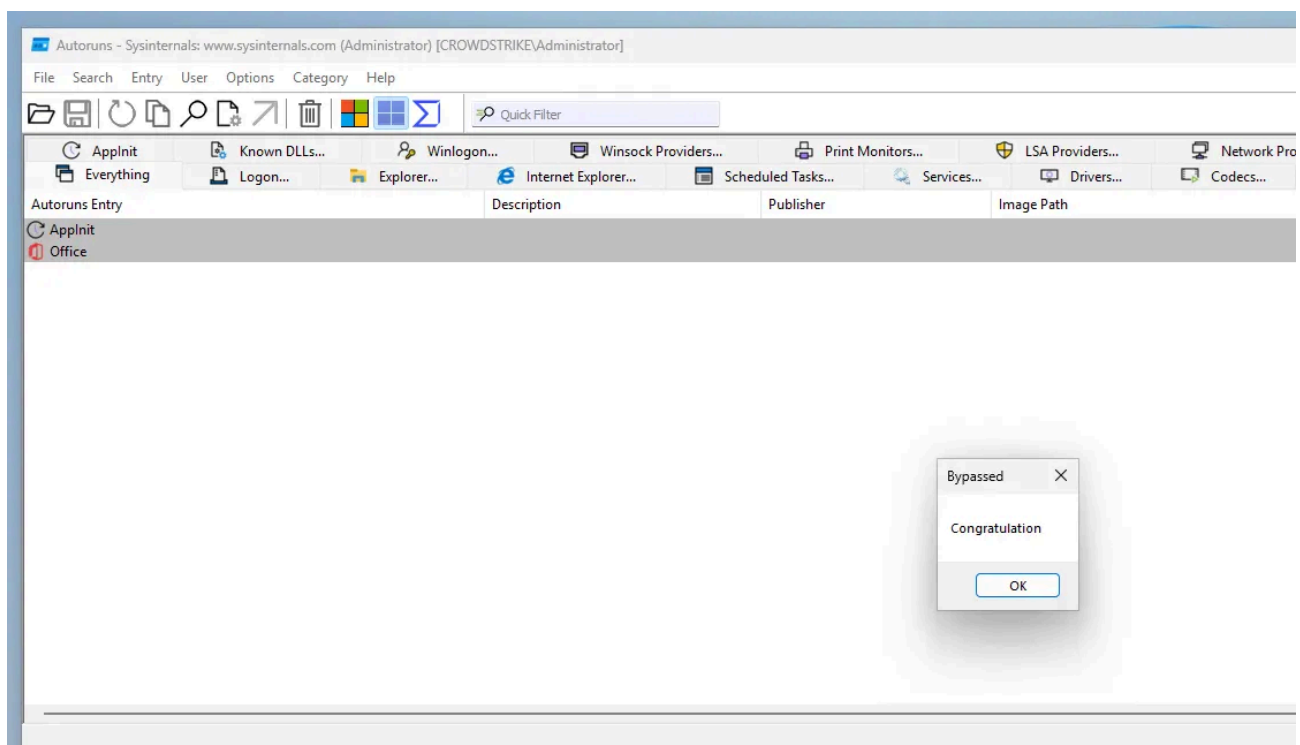
Pro tip: The `--bypass-profile` option automatically applies multiple AV/EDR evasion techniques during payload generation. You can also use predefined EDR bypass profiles:

```
--bypass-profile=bypass_profiles\defender_bypass_profile.json
```

5.3: Execution

With everything in place, launch the target application `Autoruns64.exe`. You should see your payload execute while the application continues to function normally. This time, the payload is triggered when the `VerQueryValueW` function is called (in this case, it's called during the startup of `Autoruns64.exe`).

Press enter or click to view image in full size



6. What if you cannot hijack the DLL Load Order?

6.1 Proxy Application specific DLL

In our example, we used the fact the application first attempted to load a system DLL from the current directory. This is not always possible. An alternative exists if the application loads its own specific DLLs.

Imagine for example test.exe relies on test.dll in the same folder.

We can rename test.dll in test_old.dll, and create a proxy DLL to replace test.dll. In an initial access scenario, we could send a 7zip file containing the exe, the dll, and the original dll using ntfs hidden attribute.

6.2 ShellcodePack Command line

Here is how we could generate the proxy DLL given the example above.

```
shellcode_pack.exe -i "C:\msgbox.bin" -G test.dll --dll-proxy="path\to\test.dll" --  
dll-proxy-original=".\\test_old.dll" --arch=WIN_X64 --dll-run-shellcode-from-dll-  
export --dll-export=TestFct
```

By default, the malicious DLL will forward the calls to the --dll-proxy path. If you need the payload to redirect to another path, use the --dll-proxy-original=DLL_PATH option. DLL_PATH being the path to the original DLL on the target machine.

Conclusion

In this tutorial, we explored DLL sideloading and the new function proxying feature in ShellcodePack. We demonstrated how to generate proxy DLLs that can execute custom shellcode while maintaining the functionality of the target application. By leveraging these techniques, you can effectively intercept and manipulate Windows applications for various purposes, including penetration testing and red teaming.